IT2140- Database Design and Development

Year 2, Semester 1

Lab Sheet 01: SQL Data Retrieval and Filtering using SELECT, WHERE, ORDER BY, GROUP BY, and HAVING Clauses (Single-Table Queries)

Objective

At the end of this lab session, you should be able to write the SELECT command with the WHERE, ORDER BY, aggregate functions, GROUP BY, and HAVING clauses for single-table queries.

Section 1: SELECT and WHERE Clause

This section introduces the SELECT statement and the WHERE clause used to filter records in SQL queries. You will learn how to retrieve specific data from a table based on conditions, using relational and logical operators.

The **SELECT** command is used to retrieve data from database tables. The basic syntax is:

```
SELECT column1, column2
FROM table_name;
```

The **WHERE** clause allows you to apply conditions to restrict the number of records returned:

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

Relational Operators

We can use the following relational operators to define the conditions in the **WHERE** clause:

`=`, `!=` or `<>`, `>`, `<`, `>=`, `<=`

These operators help compare values and filter records based on your query conditions.

Examples:

```
SELECT ename FROM emp WHERE salary > 50000;
SELECT * FROM dept WHERE budget BETWEEN 50000 AND 2000000;
```

Additional Keywords for Filtering

We can also use the following keywords in the **WHERE** clause to create more complex conditions:

- AND, OR – Combine multiple conditions
- BETWEEN, NOT BETWEEN – Check if values fall within or outside a range
- IN, NOT IN – Match any value from a specified list
- IS NULL, IS NOT NULL – Check for missing or existing (non-null) values
- LIKE, NOT LIKE – Match text patterns (e.g., names starting with 'A')

Table 1: Operators and Keywords in WHERE Clause

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. Note: In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column
AND	displays a record if all the conditions separated by AND are TRUE

OR	displays a record if any of the conditions separated by OR is TRUE
NOT	NOT operator displays a record if the condition(s) is NOT TRUE

Section 2: ORDER BY Clause

This section explains how to sort the results of a query using the **ORDER BY** clause. You will learn how to display records in ascending or descending order based on one or more columns.

By default, SQL sorts records in **ascending** order. To change the sort order to **descending**, use the **DESC** keyword.

```
SELECT column1, column2
FROM table_name
ORDER BY column1 [ASC|DESC];
```

Section 3: Aggregate Functions

An aggregate function performs a calculation on a set of values and returns a single summarized result. These functions are essential for creating reports, summaries, and analytics from database tables. Aggregate functions ignore NULL values during computation.

When Are Aggregate Functions Used?

Using your employee database, aggregate functions can help answer questions like:

- What is the **total salary** paid to all employees?
- How many **departments** are there?
- What is the **highest salary**?
- What is the **average work percentage** assigned to employees?

Common Aggregate Functions

Function	Description
COUNT()	Function returns the number of rows that matches a specified criterion.
SUM()	Function returns the total sum of a numeric column.
MAX()	Function returns the largest value of the selected column.
MIN()	Function returns the smallest value of the selected column.
AVG()	Function returns the average value of a numeric column.
ROUND()	used to round the value up to a specific decimal place
FIRST()	Returns the first value in a column.
LAST()	Returns the last value in a column.

Examples:

Question : How many employees are there in the organization? — refer to Data_Set.sql for details.

Step 01:

You need to count how many employees exist in the organization. This can be done by counting the number of unique employee IDs (eid) in the emp table, since each employee has one unique eid.

Step 02:

Now write the SQL query to get the result:

```
SELECT COUNT(eid)
FROM emp;
```

Expected Output:

This query will return a single number — the total number of employees.

	(No column name)
1	10

You can also verify the result by using the query:

```
SELECT COUNT(*) FROM emp;
```

This will return the same number of employees, as it counts all rows in the emp table.

	eid	ename	age	salary
1	1000	Lakmal	33	90000
2	1001	Nadeeka	24	28000
3	1002	Amila	26	35000
4	1003	Nishani	28	60000
5	1004	Krishan	36	95000
6	1005	Surangi	37	22000
7	1006	Shanika	24	18000
8	1007	Amali	21	20000
9	1008	Charith	28	35000
10	1009	Prasad	40	95000

Note: COUNT(*) counts all rows, while COUNT(eid) only counts rows where eid is not NULL — but since eid is a primary key, there won't be any null values, so both queries will give the same result.

Section 4: GROUP BY Clause

The GROUP BY statement in SQL is used to **arrange identical values into groups**, especially when used with **aggregate functions** such as COUNT(), SUM(), AVG(), MAX(), or MIN().

If a particular column has the same value in multiple rows, GROUP BY will collect those rows into a **single group**, allowing you to perform calculations on each group.

Important Notes:

- GROUP BY is always used with the SELECT statement.
- It must appear **after the WHERE clause** (if one is used).
- It must come **before the ORDER BY clause** (if one is used).
- All selected columns (except those inside aggregate functions) **must be listed in the GROUP BY clause**.

Syntax:

```
SELECT column_name, AGGREGATE_FUNCTION(column2)
FROM table_name
[WHERE condition]
GROUP BY column_name
[ORDER BY column_name];
```

Example:

Question: Count the number of employees in each age group. **— refer to Data_Set.sql for details.**

Step 01:

View the table to understand column structure:

```
SELECT * FROM emp;
```

Step 02:

Use GROUP BY on the age column:

```
SELECT age, COUNT(*) AS Number_of_Employees
FROM emp
GROUP BY age;
```

This will return the number of employees for each age found in the emp table.

	age	Number_of_Employees
1	21	1
2	24	2
3	26	1
4	28	2
5	33	1
6	36	1
7	37	1
8	40	1

Section 5: HAVING Clause

The HAVING clause allows you to filter groups created by the GROUP BY clause based on aggregate function results. It is used because the WHERE clause cannot filter aggregated data.

- WHERE filters rows before grouping.
- HAVING filters groups after aggregation.

If a SQL query uses both, the order is:

WHERE → GROUP BY → HAVING → ORDER BY

Syntax:

```
SELECT column1, aggregate_function(column2)
FROM table_name
[WHERE condition]
GROUP BY column1
HAVING aggregate_function(column2) condition
[ORDER BY column1];
```

- column1, column2: columns to select
- aggregate_function: e.g., SUM(), COUNT(), AVG(), etc.
- WHERE condition: filter rows before grouping
- HAVING condition: filter groups after grouping

Example:

Question: Find how many employees work in each department but only display departments that have **more than 2 employees**. — refer to Data_Set.sql for details.

Step 01:

Check the works table to understand employee context:

```
SELECT * FROM works;
```

Expected Output:

	eid	did	pct_time
1	1000	Admin	40
2	1000	ITSD	50
3	1001	Admin	100
4	1002	Academic	100
5	1003	Academic	30
6	1003	Admin	20
7	1003	ITSD	45
8	1004	Admin	60
9	1004	Finance	30
10	1006	Finance	45
11	1006	Maintenance	52
12	1008	Finance	35
13	1008	ITSD	30
14	1008	Maintenance	30
15	1009	Admin	100

Step 02:

Use GROUP BY and HAVING to count employees per department and filter by count:

```
SELECT did, COUNT(eid) AS Number_of_Employees
FROM works
GROUP BY did
HAVING COUNT(eid) > 2;
```

Expected Output:

	did	Number_of_Employees
1	Admin	5
2	Finance	3
3	ITSD	3